1 Writing the Kernel Module

With the example `uio_cif.c` example it is very straightforward to figure out how to write the Kernel Module.

First, we have to unregister the default serial driver for the Mini-UART. The default driver defines the `compatible` string to be `brcm,bcm2835-aux-uart`.

We overwrite the `compatible` string in the device tree file to

```
&uart1 {
    compatible = "uio_uart";
    ...
}
```

With this change, the default driver won't register, and we are free to provide our own.

Probing the device is quite simple as well. To register a UIO Device we must fill the `struct uio_info`. We provide the handle to the custom `irqcontrol()` function here and an implementation of `open()` so interrupts are only enabled once the device is used. The corresponding `release()` function disables interrupts once the driver stops using the device file.

For memory, we will have a look at the `pdev->resource[0]`. It contains all needed information to populate `struct uio_mem` and is given by the dts. We align the memory to the page boundary, meaning it will start from `0x07e215000` and not `0x07e215040`. why?

Local state can be maintained through the `priv` pointer in the `uio_info struct`. We use it to keep track of enabled / disabled interrupts.

1.1 The Interrupt Handler

Our hardware needs *some* action to be performed after each interrupt: Either acknowledging it by reading the byte or disabling interrupts from the hardware. This limitation is imposed by the hardware as "The Interrupt Line is asserted whenever the Receive FIFO holds at least 1 byte / Transmit FIFO is empty".

If we were not to do anything, the interrupt would be asserted before our userspace driver is scheduled and could handle the byte leading to our kernel interrupt handler being called all the time.

We present two approaches to handle this limitation: Disabling Interrupts and using the `SCRATCH` register.

1.1.1 Disabling Interrupts

This approach is the most straightforward: Simply disable receive / transmit interrupts once one occurs and return `IRQ_HANDLED`. This will give the userspace part of the driver enough time to be scheduled and read the `IO` register. Once the byte has been read, it enables interrupts by writing a `1` to `/dev/uio0`.

The following shows a flow graph of the interrupt handling routine

Figure 1: Interrupt Handling when disabling Interrupts

1.1.2 The `SCRATCH` Register

The second approach we want to present is using the `SCRATCH` register. It "is a single byte of temporary storage" provided by the UART. The Kernel will, upon entering the interrupt handling routine, read the available byte from the `IO` register and save it to the `SCRATCH` register.

A limitation imposed by this approach is that we are only ever able to handle a single byte per interrupt. With the previous approach, the userspace could read check the `AUX_MU_STAT_REG` register for the FIFO level and read as many bytes as there are available. With this approach, the kernel needs to place the byte from the `IO` register into the `SCRATCH` register, meaning a context switch into kernel-space is necessary for every byte.

The following shows a flow graph of the interrupt handling routine

Figure 2: Interrupt Handling when buffering in the `SCRATCH` register

As we first step, we provide a reference implementation with polling. This is the baseline in terms of performance as the CPU speed will be the bottleneck for data throughput. Next, we want to investigate the improvements we will be able to achieve with interrupts, specifically IRQ. Finally, we provide a patch for the Linux Kernel enabling the use of FIQ in UIO. The differences in latency and throughput will be of great interest.

1.2 Enabling the UART

Adding the following lines to `/boot/firmware/config.txt` enables the mini-UART:

```
enable_uart=1
core_freq=250
```

Additionally, the linux console has to be disabled with the `raspi-config` binary.[1] Of course, the custom dtb has to be booted by the kernel to recognize the mini-uart as a uio device

1.3 Measurements

- How to measure?

1.4 Details

Second Raspi

- NFS Booting

[1] https://www.raspberrypi.com/documentation/computers/configuration.html#disabling-the-linux-serial-console